LANGUAGE MODEL DILIGENCE

A Framework for AI Process Governance

Published by Mossrake Group, LLC
June 2026

Organizations are deploying AI models into processes where wrong answers have real consequences. This paper introduces Language Model Diligence – the practice of systematically understanding that exposure – and presents a framework grounded in established practices from quantitative finance, pharmaceutical manufacturing, and avionics safety engineering.

Executive Summary

The rapid adoption of large language models across industries has created a governance gap. Governance hasn't kept pace with deployment. Many organizations don't have systematic visibility into how language models are deployed and operate, or reliable mechanisms to verify outputs and recover from failures.

The traditional tools of software quality assurance – unit tests, integration tests, staged rollouts – were designed for deterministic systems. They assume that identical inputs produce identical outputs, that boundary conditions can be enumerated, and that a passing test suite provides meaningful coverage guarantees. None of these assumptions hold for large language models, which are stochastic by nature, whose behavior can change without notice when providers update their models, and whose input space is effectively unbounded.

Language Model Diligence is a framework for addressing this gap. Rather than attempting to certify that an AI model is correct – an impossible standard for a probabilistic system – the framework asks whether an organization has sufficient coverage in place to make informed governance decisions about the models it deploys.

The framework consists of five questions. The first establishes the consequence level: how much does it matter if this model produces a wrong output? This consequence classification then determines the rigor threshold for the remaining four questions, each of which measures an independent coverage dimension: Identity (do you know what model is running?), Drift (would you know if it changed?), Verification (is output quality checked before it matters?), and Recovery (if something gets through, how fast is it caught?).

The resulting assessment produces one of three tiers – Calibrated, Partial, or Uncertain – describing the relationship between what's at stake and what the organization can see. These tiers measure the gap between consequence and coverage – what this framework calls exposure. An organization assessed as Calibrated has coverage proportional to its consequence level; whether it governs well is a separate question.

This paper presents the intellectual foundations of Language Model Diligence, drawn from model risk management in quantitative finance, process validation in pharmaceutical manufacturing, and design assurance in avionics safety engineering. It describes the framework in detail and demonstrates its application through worked examples.

The Problem

Governance hasn't kept pace with how organizations deploy technology. The gap is structural, not one of awareness or intent.

When a company deploys traditional software – a billing system, a routing algorithm, a rules engine – the behavior of that software is deterministic and inspectable. Given the same input, it produces the same output. When it fails, the failure can be traced through code to a specific defect. When it is updated, the changes are enumerated in release notes. The entire edifice of software quality assurance rests on these properties: test a representative set of inputs, verify the outputs, and have high confidence that the system will behave as expected in production.

Large language models have none of these properties. They are stochastic: the same input may produce different outputs across invocations. They are opaque: even the providers cannot fully explain why a model produces a particular response. They are mutable: providers routinely update models behind stable API endpoints, sometimes with notice and sometimes without. And their input space is effectively unbounded: unlike a function with defined parameters and edge cases, a language model accepts arbitrary natural language, making exhaustive testing impossible.

Despite these fundamental differences, organizations are deploying language models into processes with real consequences. Models draft customer communications, screen regulatory filings, triage support requests, approve transactions, and generate analyses that inform business decisions. In many cases, the model's output is the decision – there is no downstream check before the output reaches its consequence.

The governance gap is not that organizations are unaware of risk. Most acknowledge, in general terms, that AI systems can produce incorrect outputs. The gap is in systematic visibility: which models are running where, what would happen if a model produced a wrong answer, whether anyone would notice, and how quickly. Without this visibility, governance decisions – where to invest in oversight, which processes need tighter controls, what level of monitoring is proportional to the stakes – cannot be made on an informed basis.

This is not a technology problem awaiting a technology solution. It is a coverage problem. The prerequisite for governing an AI model is knowing what it is doing and having the mechanisms to verify and recover. Language Model Diligence is a framework for building that coverage systematically.

Why Software Testing Metaphors Fail

The instinct to apply software testing practices to AI models is understandable but misguided. The failure is not one of execution but of category. Software testing and AI model governance are addressing fundamentally different types of systems.

Determinism and Reproducibility

Software testing depends on a property so fundamental it is rarely named: reproducibility of behavior across identical inputs. Test a function, record the output, and know the answer forever. This property enables regression testing – the practice of re-running previous tests after a change to verify that existing behavior is preserved. Regression testing is the backbone of software quality because it converts past testing effort into ongoing assurance.

Language models do not provide this property. Even at low temperature settings, outputs exhibit variation. More significantly, model providers can change what a given API endpoint returns without changing the endpoint identifier. An organization that tested a model last month and received satisfactory results has no guarantee that the same model – or even the same version of the model – is responding today.

Bounded Input Spaces

Effective software testing relies on the ability to identify boundary conditions and equivalence classes. A function that processes dates can be tested at year boundaries, month boundaries, leap years, and invalid inputs, achieving high coverage with a finite test set. The confidence that untested inputs in the interior will behave correctly follows from the deterministic structure of the code.

Language models accept arbitrary natural language. There are no boundary conditions to enumerate, no equivalence classes to define, and no structural basis for believing that tested inputs predict untested behavior. A model that performs flawlessly on a thousand carefully designed test cases may fail on the thousand-and-first in a way that no test designer would have anticipated.

Silent Model Changes

When traditional software is updated, the update is an event: a new version is released, changelogs are published, and the organization decides when to adopt it. The previous version continues to function unchanged until explicitly replaced.

Language model providers operate differently. Models are accessed through API endpoints that abstract away the underlying implementation. A provider may update a model's weights, fine-tuning, or system behavior and the consuming organization may receive no notification. The API contract – send a prompt, receive a completion – remains unchanged while the behavior behind it shifts. Organizations that built processes around a model's observed behavior may find that behavior has changed without any corresponding change in their own systems.

The Implication

These three properties – stochastic outputs, unbounded inputs, and silent changes – mean that the software testing paradigm of “test it, ship it, retest when something changes” cannot provide meaningful assurance for AI model deployments. What is needed instead is continuous coverage: an ongoing understanding of what model is running, whether its behavior is stable, whether its outputs are being checked, and how quickly problems would surface. This is not testing. It is diligence.

Intellectual Lineage

The challenge of governing AI models – systems whose behavior is probabilistic, whose internal mechanisms are not fully understood, and whose outputs cannot be exhaustively tested – is not without precedent. Three established disciplines have developed rigorous frameworks for governing systems with similar properties.

Model Risk in Quantitative Finance

The financial services industry has governed probabilistic models for decades. Value-at-Risk models, credit scoring algorithms, and pricing models are all systems whose outputs are probabilistic, whose accuracy degrades over time as market conditions change, and whose failures can have severe consequences.

In 2011, the Federal Reserve and the Office of the Comptroller of the Currency jointly published SR 11-7, the foundational regulatory guidance on model risk management. SR 11-7 established principles that translate directly to AI governance: models should be independently validated, their performance should be monitored over time, and the rigor of governance should be proportional to the model’s materiality and the consequences of its failure.

Emanuel Derman, the physicist turned quantitative analyst whose work on model risk predates SR 11-7, articulated the core insight: *models are metaphors, not reality, and the danger comes from forgetting that distinction.* Language models present this danger in an amplified form. Their outputs are fluent, confident, and formatted like authoritative answers – making it particularly easy to forget that they are probabilistic suggestions from a system that does not understand what it is saying.

Process Validation in Pharmaceutical Manufacturing

The pharmaceutical industry governs processes whose outputs cannot be individually verified. A manufacturing batch may contain millions of tablets; testing every one would consume the batch. Instead, pharma developed a framework for ensuring that the process itself is reliable enough that individual output testing is unnecessary.

The core structure is a three-stage validation lifecycle. Installation Qualification (IQ) verifies that the system is set up correctly. Operational Qualification (OQ) verifies that it operates within specified parameters under controlled conditions. Performance Qualification (PQ) verifies that it

performs acceptably under real production conditions. These stages are sequential gates: you do not proceed to the next until the previous is satisfied.

Two additional pharmaceutical practices are particularly relevant. Process Analytical Technology (PAT) is the FDA-endorsed practice of monitoring critical quality attributes during manufacturing rather than relying on end-of-line testing. The parallel to real-time AI model monitoring is direct: catching a problem during the process is cheaper than catching it in the output. And ICH Q9, the international standard for Quality Risk Management, introduced a three-dimensional risk framework that evaluates severity, probability, and detectability. The third dimension – detectability, the likelihood that a failure will be caught before it causes harm – is the dimension most AI governance frameworks omit and the one that matters most.

Design Assurance in Avionics

DO-178C, the standard for software in airborne systems, takes a consequence-based approach to governance rigor. Software is classified into five Design Assurance Levels (A through E) based on the consequences of failure, from catastrophic to no effect. The verification objectives – testing, analysis, review – scale with the level. Level A software requires exhaustive testing and formal verification. Level E software requires almost nothing.

The principle is simple and powerful: the rigor of governance should be proportional to the consequences of failure. A failure in a flight control system and a failure in an in-flight entertainment system have the same technical complexity but radically different governance requirements. This principle is the foundation of the Language Model Diligence consequence classification.

Synthesis

The Mossrake Language Model Diligence framework draws from all three traditions. From quantitative finance, the recognition that probabilistic models require ongoing monitoring and that governance rigor should scale with materiality. From pharmaceutical manufacturing, the validation lifecycle, the emphasis on process observation over output testing, and the critical dimension of detectability. From avionics, the principle that consequence determines the appropriate level of rigor. These are not novel concepts being applied to a new domain. They are proven governance principles being extended to a class of technology that urgently needs them.

The Mossrake Language Model Diligence Framework

The framework is administered through the Mossrake Language Model Diligence assessment, a structured instrument for evaluating the coverage posture of a single AI model deployed in a defined process. It consists of a framing statement and five questions.

The Statement

Every assessment begins with a framing statement that establishes the context. The respondent identifies the model and its role in the process: what the model does, what inputs it receives, and what actions are taken based on its output. This description appears in the assessment report and serves as the basis for evaluating the answers that follow.

The assessment covers a single model performing a defined function. Processes involving multiple models, pipelines, or agentic architectures require facilitated assessment due to the additional complexity of inter-step dependencies and cascade risk.

Question 1: Consequence

How much does it matter if this model produces a wrong output?

The first question is not scored as a dimension. It is the lens through which all subsequent answers are evaluated. The respondent classifies the consequence of model failure on a five-level scale:

Level	Label	Description
1	Negligible	Wrong outputs would have little or no practical effect – no decisions, commitments, or downstream processes depend on accuracy.
2	Inconvenient	Wrong outputs cause rework or minor inefficiency with no impact beyond the team.
3	Consequential	Wrong outputs could affect other teams, customers, or decisions, but damage is correctable.
4	Damaging	Wrong outputs could cause financial loss, reputational damage, or compliance issues requiring remediation.
5	Critical	Wrong outputs could cause regulatory action, legal liability, safety consequences, or irreversible harm.

At Level 1, the assessment concludes. The model does not warrant formal diligence. This self-selection mechanism ensures that every completed assessment represents a process with genuine stakes, improving the quality of benchmark data and focusing organizational attention where it matters.

At Level 2 and above, the consequence classification determines the coverage threshold for the four dimensions that follow.

Questions 2–5: Four Coverage Dimensions

The remaining four questions each measure an independent coverage dimension – the visibility, controls, and recovery mechanisms in place around the model. Each requires two responses: a self-assessment on a dimension-specific 1–5 scale, and a verbatim explanation describing the current state in the respondent's own words. The numeric score provides structured, comparable data across assessments. The verbatim provides evidence and context. Critically, coverage scores are dimensionless without the consequence classification. A score on any dimension is not inherently good or bad – it becomes meaningful only when evaluated against the stakes established in Question 1.

Q2: Identity – Do you know what's running?

Do you know exactly which model and version is running this process?

Identity is a point-in-time fact about the current state. The self-assessment scale ranges from Unknown (the organization does not know what model is running) through General (provider known, model unknown), Approximate (model family known, version not pinned), Specified (exact model and version in the implementation), to Verified (model and version specified and confirmed through a validation mechanism).

Many organizations discover during this question that they cannot identify the specific model serving their process. They may be calling a wrapper API, using a framework default, or relying on an endpoint whose underlying model the provider can change. The inability to answer this question with certainty is itself a significant finding.

Q3: Drift – Would you know if it changed?

If the model or its behavior changed, would you know?

Drift is temporal awareness – the ability to detect behavioral change over time. It is independent of Identity: an organization could detect behavioral drift without knowing the model identifier (through output regression testing), or could know the identifier but have no mechanism to detect a silent update.

The scale ranges from Unmonitored (no mechanism to detect change) through Anecdotal (someone might notice), Informal (ad hoc monitoring or changelog subscriptions), Monitored (defined quality metrics or automated checks), to Tested (automated behavioral regression tests against defined acceptance criteria). This dimension draws directly from the pharmaceutical concept of stability testing – the practice of periodically verifying that a product remains within specification over time, regardless of whether a change is known to have occurred.

Q4: Verification – Is anyone checking the work?

Is output quality checked before the output reaches its end use?

Verification is the preventive control – anything that evaluates the model's output before it reaches its consequence. This could be a human reviewer, an automated quality gate, a rule-based filter, or a sampling protocol. The key word is "before."

The scale ranges from None (output goes directly to end use) through Incidental (someone sees it in passing), Sampled (periodic spot-checks), Systematic (automated gate or defined process for every output), to Qualified (every output reviewed by a person with relevant domain expertise).

Verification can be expensive, and is consequently one of the first dimensions to be minimized under cost pressure. This makes it particularly important to assess honestly. This dimension surfaces a common and consequential pattern: the rubber-stamp review. A human technically sees the output but does not meaningfully evaluate it. The distinction between a score of 2 (incidental exposure) and 5 (qualified review) is not about whether someone sees the output. It is about whether the review has teeth.

Q5: Recovery – If something gets through, how fast is it caught?

If a bad output got through, how quickly would it be caught?

Recovery is the detective control – the safety net for when verification fails or does not exist. It is independent of verification: an organization could have strong verification and no recovery (the reviewer catches most problems, but what slips through is never found), or no verification and strong recovery (outputs go to production unchecked, but automated monitoring catches anomalies within hours).

The scale ranges from Undetectable (bad outputs would likely not be caught unless externally flagged) through Eventual (caught during periodic audits, weeks later), Lagging (caught within days), Prompt (caught within hours), to Immediate (real-time monitoring or rapid feedback within minutes). The critical measure is whether the detection window is proportional to the rate at which damage accumulates – a concept this framework terms consequence velocity.

Orthogonality

The four dimensions are designed to be independent. Each can be strong while the others are weak, and each addresses a distinct failure mode:

Dimension	Failure Mode	Independence Example
Identity	You don’t know what’s producing your outputs.	You specify the model version but have no drift detection. Identity is strong; Drift is weak.
Drift	The model changed and you didn’t know.	Behavioral tests catch changes, but you don’t know which model is behind the endpoint. Drift is strong; Identity is weak.
Verification	A bad output wasn’t caught before it mattered.	Every output is reviewed, but what the reviewer misses is never found. Verification is strong; Recovery is weak.
Recovery	A bad output that got through wasn’t caught afterward.	No pre-check exists, but monitoring flags anomalies within hours. Recovery is strong; Verification is weak.

This orthogonality ensures that no dimension is redundant and no gap is masked by strength in another area. A process with excellent Verification but no Recovery has a real and distinct vulnerability: anything the verification step misses becomes invisible. The assessment captures this precisely because the dimensions are scored independently.

Scoring and Tiers

The assessment produces one of three tiers. These tiers describe the relationship between consequence and coverage – between what's at stake and what the organization can see. The gap between the two is what this framework calls exposure.

Three Tiers

Calibrated. Coverage is proportional to consequence across all four dimensions. The organization knows what model is running, would detect changes, checks output quality, and has a recovery mechanism for what gets through. This does not mean the model is safe or well-governed – it means coverage matches what the stakes demand. Minimal exposure.

Partial. Coverage falls short of what consequence demands in one or more dimensions. Material exposure exists. Specific gaps are identified and prioritized. Most organizations will land here on their first assessment. The value is knowing which gaps matter most.

Uncertain. Coverage cannot be determined on one or more dimensions, so exposure is unknown. This is not a judgment about competence. It is a factual description of an information gap. The remediation path is building basic visibility before exposure can be assessed.

Exposure: The Gap Between Consequence and Coverage

Each coverage dimension is scored independently against the consequence level established in Question 1. The same score can represent zero exposure at one consequence level and significant exposure at another. Coverage scores are dimensionless without consequence – they become meaningful only in the relationship.

Consequence Level	Coverage Threshold	Exposure Threshold
2 – Inconvenient	Score of 2+ on Q2–Q5 generally sufficient.	Score of 1 on any dimension is a finding.
3 – Consequential	Score of 3+ on Q2–Q5 expected.	Score of 2 on any dimension is a gap.
4 – Damaging	Score of 4+ on Q2–Q5 expected.	Score of 3 or below is a gap.
5 – Critical	Score of 4–5 on all dimensions required.	Score of 3 on any dimension is a gap.

This structure makes the assessment self-calibrating. A meeting summarizer (Consequence Level 2) and a financial approval model (Consequence Level 4) can have identical coverage

scores and receive different tiers, because the same gap represents more exposure when the stakes are higher. A framework that imposes the same rigor on every AI deployment is asking too much in some places and too little in others. Language Model Diligence asks whether coverage is proportional to consequence.

Worked Examples

The following examples illustrate how the same five questions produce different outcomes for models deployed at different consequence levels and with different coverage levels.

Example 1: Internal Meeting Summarizer

Process: A language model summarizes meeting transcripts for the engineering team. Summaries are posted in an internal channel.

Question	Score	Response Summary
Q1: Consequence	2	Wrong summaries cause minor rework. No external impact.
Q2: Identity	3	Team uses “Claude Sonnet” but has not pinned a version.
Q3: Drift	2	No formal monitoring. Team would notice if summaries degraded noticeably.
Q4: Verification	2	Meeting attendees see summaries and would catch major errors.
Q5: Recovery	3	Errors caught within a day or two at the next standup.

Result: Calibrated. At Consequence Level 2, scores of 2–3 across all dimensions meet the threshold. Coverage is proportional to consequence. Minimal exposure. Formal monitoring would not be a productive investment for this process.

Example 2: Customer Support Draft Generator

Process: A language model generates draft responses to inbound support tickets. Drafts go to an agent queue for review before sending to the customer.

Question	Score	Response Summary
Q1: Consequence	3	A bad response reviewed and sent could damage the customer relationship.
Q2: Identity	4	Implementation specifies claude-sonnet-4-6.
Q3: Drift	2	No drift monitoring. Team relies on agents to notice quality changes.
Q4: Verification	4	Every draft is reviewed by a support agent before sending.
Q5: Recovery	3	Customer complaints surface within days. CSAT surveys within a week.

Result: Partial. At Consequence Level 3, the coverage threshold is a score of 3 or above. Q3 (Drift) scores a 2, creating exposure on that dimension. The organization has no mechanism to detect if the model's behavior changes, relying entirely on the human verification step to catch degradation. The priority gap is drift detection. The recommendation is to implement output quality monitoring or behavioral checks that would surface model changes independently of human review.

Example 3: Invoice Auto-Approval

Process: A language model reviews incoming invoices against purchase orders and auto-approves matched items for payment up to $5,000.

Question	Score	Response Summary
Q1: Consequence	4	Incorrect approvals create direct financial loss and potential audit findings.
Q2: Identity	3	Team knows the model family but uses an API gateway that could route to different versions.
Q3: Drift	1	No monitoring of any kind. No changelog review.
Q4: Verification	1	Approved invoices are paid without human review.
Q5: Recovery	2	Errors found during quarterly financial review.

Result: Uncertain. At Consequence Level 4, coverage scores of 4 or above are expected. This process has scores of 1 on two dimensions (Drift and Verification) and a score of 2 on Recovery. The gap between consequence and coverage is severe – significant exposure across three of four dimensions. The quarterly detection window means that incorrect approvals could accumulate for months. This assessment surfaces an urgent need to implement pre-payment verification and drift monitoring before any further auto-approvals are processed.

Example 4: Regulatory Compliance Screening

Process: A language model screens marketing materials against regulatory guidelines before publication. The model identifies potential violations and generates a summary for the compliance officer.

Question	Score	Response Summary
Q1: Consequence	5	A missed violation could result in regulatory action and significant fines.
Q2: Identity	5	Exact model and version pinned. Deployment script verifies the model identifier on every startup.
Q3: Drift	4	Weekly behavioral regression tests against a library of known compliant and non-compliant materials.
Q4: Verification	5	Every screening result is reviewed by a certified compliance officer. Model output is advisory only.

Q5: Recovery	4	All published materials are re-screened monthly. Compliance hotline for external reports.

Result: Calibrated. At Consequence Level 5, scores of 4–5 are required across all dimensions. This process meets the threshold on every dimension. The model is identified and verified, drift is actively tested, every output is reviewed by a qualified professional, and a recovery mechanism exists for anything that gets through. Coverage is proportional to consequence. Minimal exposure.

The Regulatory and Economic Landscape

Regulatory Momentum

The European Union's AI Act, which classifies AI systems into risk tiers and imposes governance requirements proportional to the tier, establishes the first comprehensive regulatory framework for AI governance. Its risk-tiered approach mirrors the consequence-based classification at the heart of Language Model Diligence. Organizations subject to the AI Act will need to demonstrate that their governance posture is proportional to their risk tier – a requirement the Mossrake Language Model Diligence assessment is designed to address.

In the United States, the growing adoption of software bills of materials (SBOMs) following executive orders on cybersecurity suggests a trajectory toward similar transparency requirements for AI. A model bill of materials – an enumeration of which models are deployed where, in what capacity, at what risk level – is a natural extension of the SBOM concept. Language Model Diligence produces this inventory as a byproduct of the assessment process.

Economic Pressure on Model Portfolios

The economics of AI deployment are creating a governance challenge of their own. As organizations recognize that AI labor costs are directly fungible against human labor costs, they are increasingly optimizing their model portfolios for cost. This means using premium models for high-consequence tasks and less expensive models – including open-source and internationally sourced models – for routine work.

This heterogeneous model portfolio creates jurisdictional supply chain risk. Models served from providers subject to different regulatory environments, different data governance norms, and different transparency obligations present risks that go beyond model quality. An organization using a mix of domestic and international models needs to understand not only which model is running each process, but what governance and transparency regime that model operates under. The Identity dimension of Language Model Diligence – do you know exactly which model is running? – takes on additional significance in this context.

Pipelines and Multi-Model Architectures

The emerging category of agentic AI systems – in which multiple models collaborate, share context, and make sequential decisions with limited human oversight – represents the frontier of governance complexity. In these architectures, a wrong output from one model can propagate through subsequent models, compounding errors or creating failures that are difficult to attribute to any single step.

The Mossrake Language Model Diligence framework, as presented in this paper, addresses single-model processes. The extension to multi-model architectures is an active area of development, requiring additional concepts including critical-node identification, inter-step analysis, and cascade risk assessment. Organizations operating agentic systems are encouraged to engage with Mossrake directly for facilitated assessment.

Conclusion

Every organization deploying AI models into processes with real consequences should be able to answer five questions about each model: how much the stakes matter, what model is running, whether they would know if it changed, whether outputs are checked before they matter, and how quickly problems would be caught.

Many organizations find they cannot answer three of these questions with confidence. That gap is not a failure of technology or intent. It is a failure of coverage – the basic prerequisite for governance.

Language Model Diligence provides a structured, repeatable approach to closing that gap. It does not require new technology, new organizational structures, or new regulatory mandates. It requires the willingness to ask honest questions and act on the answers.

The framework is designed to be proportional: a low-stakes model warrants light coverage; a high-stakes model warrants rigorous coverage. It is designed to be actionable: every completed assessment identifies specific gaps and prioritized next steps. And it is designed to be transparent: the five questions, the three tiers, and the scoring logic are openly published so that any organization can begin its Language Model Diligence practice today.

The first step is the assessment. It takes less time than the risk warrants.

To take the Mossrake Language Model Diligence assessment, visit:

mossrake.ai/language-model-diligence